

**Object-Oriented Analysis and UML-Based Design of
an**

**Autonomous Drone-Based Disaster Response and
Rescue System**

NAME: S.Md.Irfan

REGNO: 192373001

COURSE CODE: CSA 1109

**Object-Oriented Analysis and UML-Based Design of an
Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)**

Field	Details
Course Code & Name	CSA1109 – Object Oriented Analysis and Design
Assignment Title	Object-Oriented Analysis and UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)
Course Outcome(s) – CO	CO1: OOAD concepts and methodologies; CO2: UML modelling of structure and behaviour; CO3: Object/class analysis and relationships
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping	SDG 3, SDG 9, SDG 11, SDG 13
Student Name / Register No.	S. Md. Irfan (192373001)

Declaration: I confirm that the work presented here is my own, and that I have followed the guidelines set out for this assessment.

Table of Contents

1. Problem Analysis and Requirement Summary
2. Actor Identification
3. Use Case Descriptions
4. Use Case Diagram
5. Object and Class Identification
6. Class Diagram
7. Sequence / Interaction Diagram
8. Activity Diagram
9. State Transition Diagram
10. Package Diagram
11. Component Diagram
12. Deployment Diagram
13. Object Responsibility Analysis
14. Design Axioms, Principles and Design Pattern Justification
15. Assumptions and Constraints
16. Requirement-to-Use Case / Class Traceability
17. Implementation / Prototype and Results
18. Reflection on Design Decisions and Learning Outcomes
19. GitHub Upload

1. Problem Analysis and Requirement Summary

At present, disaster-response operations rely on separate tools for logging incidents, controlling drones and coordinating field teams. ADDRRS brings these three functions together under a single platform.

Functional Requirements

- Log a disaster incident and mark out the zones it affects.
- Allocate autonomous drones to a mission and keep track of them.
- Plan flight routes and monitor drone status as it happens.
- Collect sensor readings, images and video streamed from drones.
- Identify victims automatically and raise emergency alerts.
- Coordinate rescue teams and follow missions through to completion.

Non-Functional Requirements

- Communication between drones and the command system must happen in real time.
- Data transmission needs to stay reliable and secure.
- The platform should scale to handle several missions running at once.
- Drone failures or lost connectivity should be handled without disrupting the system.

2. Actor Identification

Actor	Description
Emergency Coordinator	Logs incidents, marks out zones, allocates drones and oversees missions.
Drone Operator	Keeps watch on drone status and can step in with manual control when required.
Rescue Team Member	Gets alerts and reports rescue progress from the field.
System Administrator	Handles user accounts and the drone inventory.
Weather Data Service (external)	Feeds environmental data into the route-planning process.

3. Use Case Descriptions

Use Case	Actor	Description	Precondition	Postcondition
Register Disaster Incident	Emergency Coordinator	Coordinator logs a new incident, giving its type and severity.	Coordinator is signed in.	An incident record is created.
Assign Drone to Mission	Emergency Coordinator	Coordinator allocates a free drone to a rescue mission.	The incident and its zone already exist.	A mission is created and linked to a drone.
Monitor Drone Status	Drone Operator	Operator checks a drone's live battery, location and health readings.	The drone is active.	The operator has the drone's current status.

Use Case	Actor	Description	Precondition	Postcondition
Detect Victim Location	Drone Operator	System scans sensor and video feeds to locate victims within a zone.	The drone is scanning the zone.	The victim's location is recorded and an alert may be raised.
Generate Emergency Alert	System (automatic)	System raises an alert once a victim or hazard is detected.	A victim or hazard has been detected.	The rescue team is notified.
Coordinate Rescue Team	Rescue Team Member	Team member receives mission details and reports on rescue progress.	An alert has been issued.	Mission status is updated.

4. Use Case Diagram

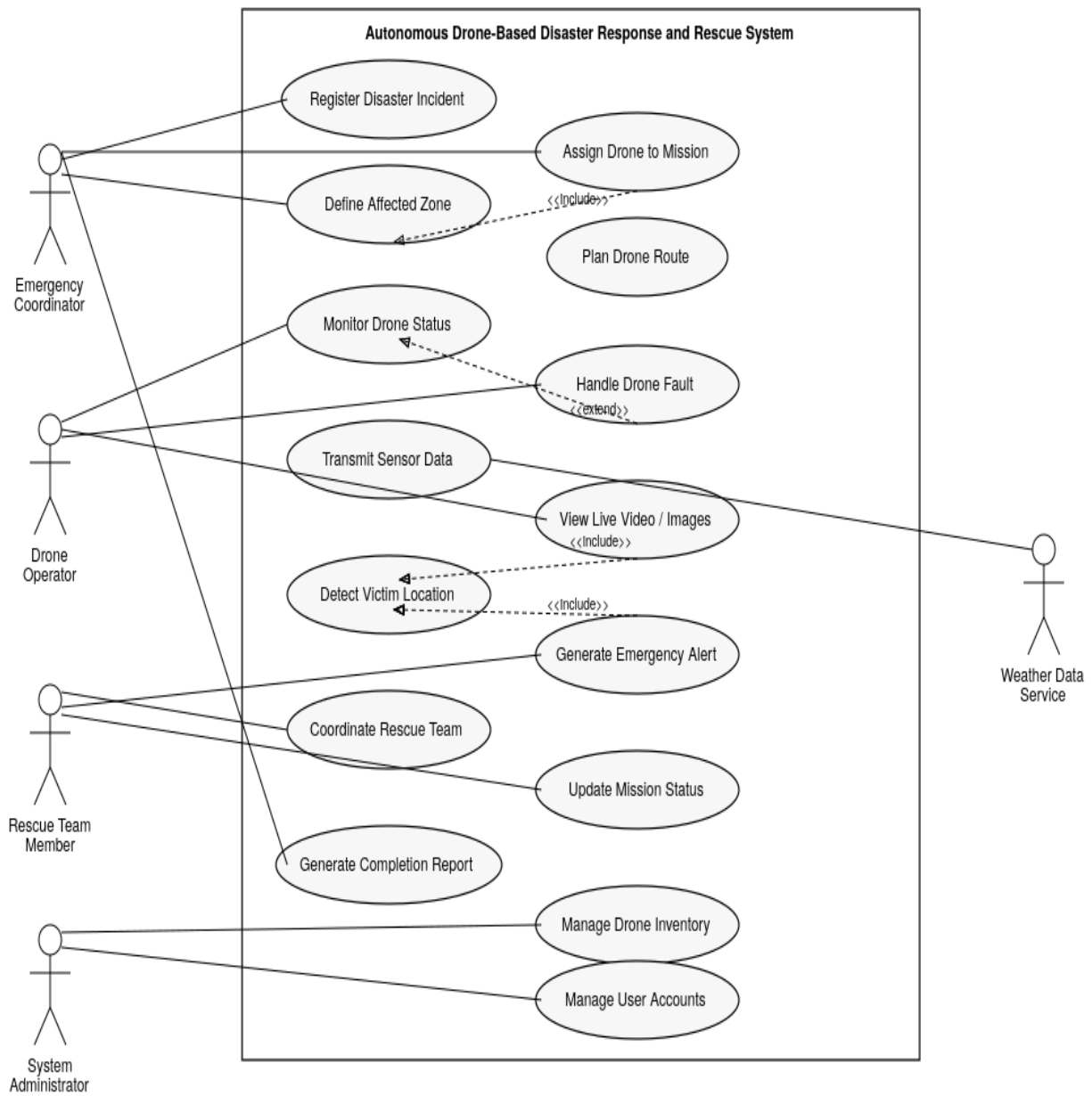


Fig. 1 — Use Case Diagram of ADDRRS showing actors, use cases, include/extend relationships and the system boundary.

5. Object and Class Identification

Class	Key Attributes	Key Methods
User (abstract)	userId, name, username	login(), logout()
EmergencyCoordinator	coordinatorId, designation	registerIncident(), assignDrone()
DroneOperator	operatorId, certificationLevel	monitorDrone(), overrideControl()
DisasterIncident	incidentId, type, severity	registerIncident(), closeIncident()
Zone	zoneId, boundaryCoordinates, riskLevel	defineZone(), updateRiskLevel()
Drone	droneId, batteryLevel, status	takeOff(), land(), returnToBase()
RescueMission	missionId, status, startTime	assignMission(), completeMission()
Victim	victimId, location, status	updateStatus()
Alert	alertId, type, priority	generateAlert(), notifyTeam()
SensorData	dataId, type, value	captureData(), transmitData()

6. Class Diagram

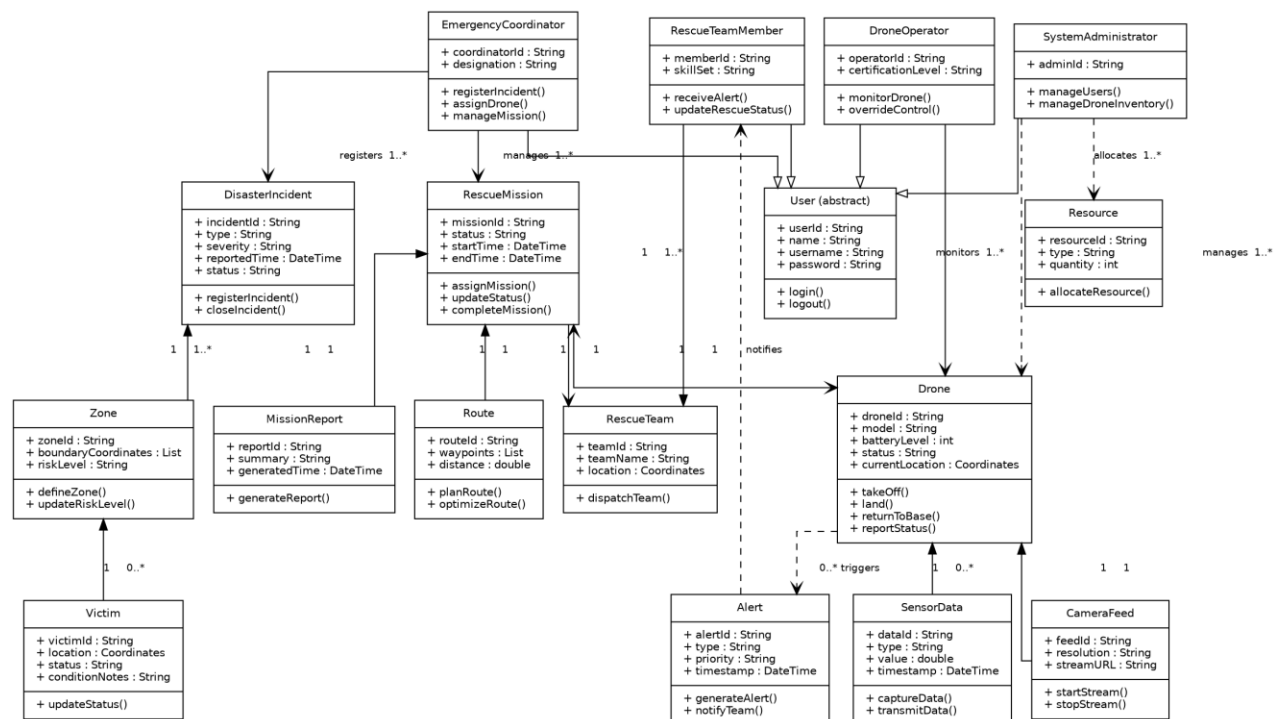


Fig. 2 — Class Diagram showing inheritance within the User hierarchy, together with composition, aggregation and association relationships.

7. Sequence / Interaction Diagram

The sequence below follows the steps taken to register an incident and then assign a drone to the resulting mission.

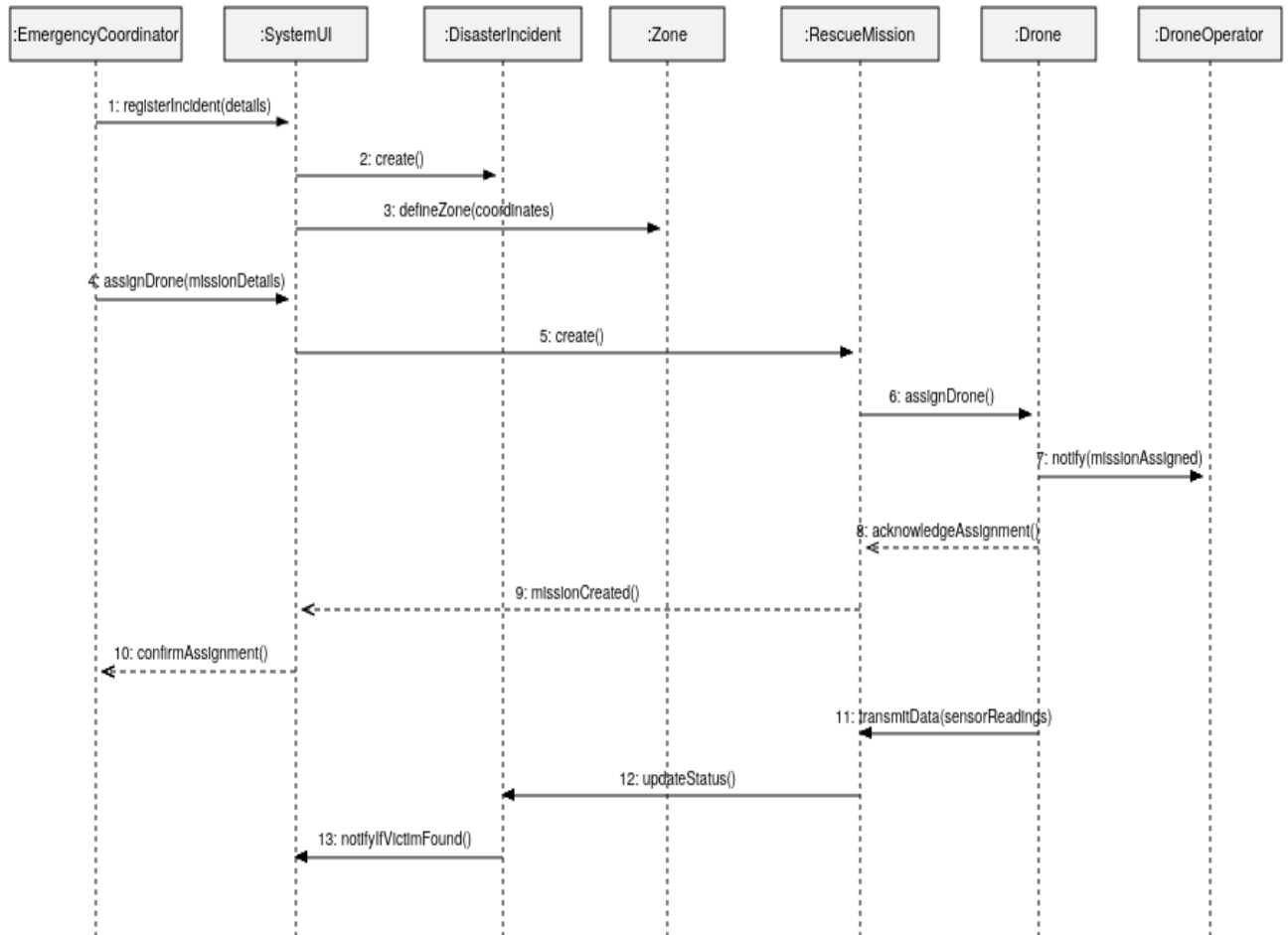


Fig. 3 — Sequence Diagram: Register Incident and Assign Drone to Mission.

8. Activity Diagram

This activity diagram lays out a drone's mission-execution workflow, covering victim detection and battery checks along the way.

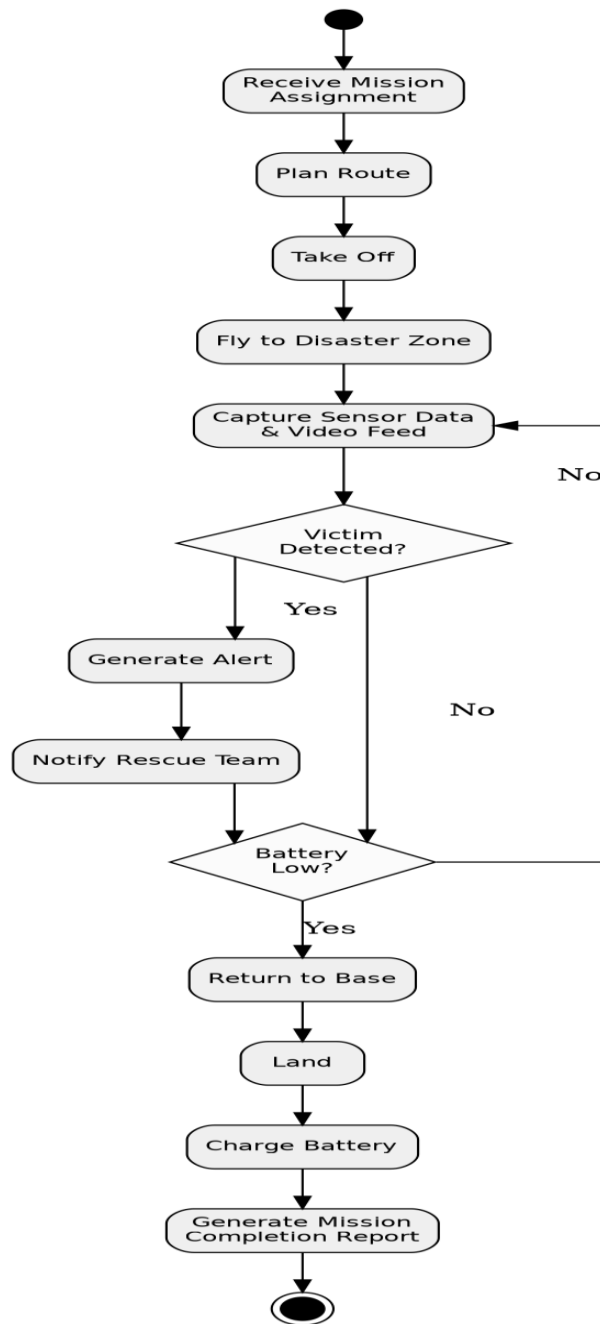


Fig. 4 — Activity Diagram: Drone Mission Execution.

9. State Transition Diagram

The diagram below shows the states a Drone object moves through over its lifecycle.

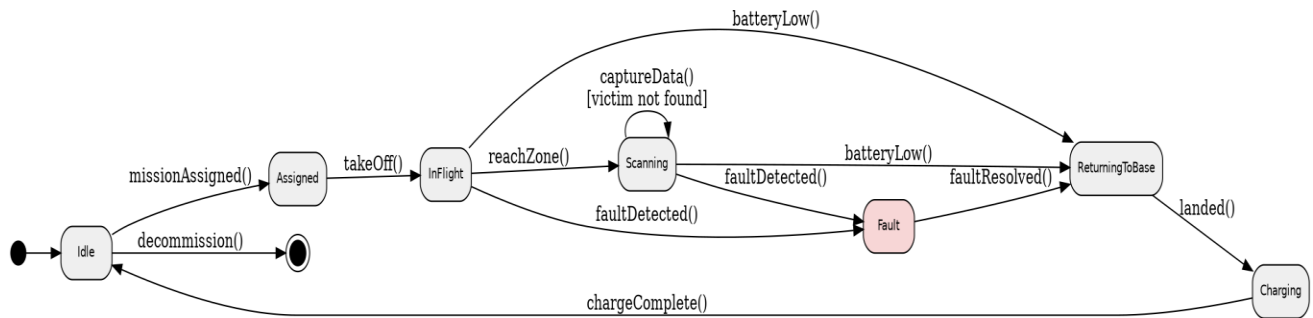


Fig. 5 — State Transition Diagram for the Drone class.

10. Package Diagram

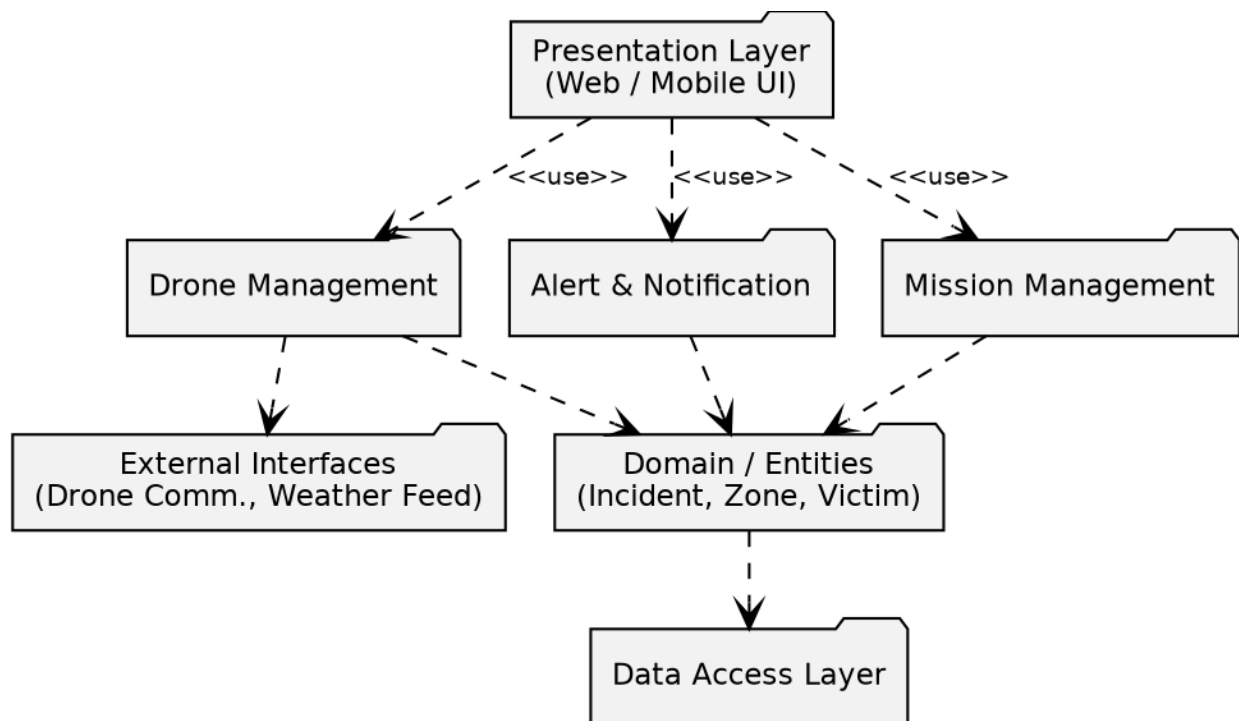


Fig. 6 — Package Diagram showing the system's modular structure and the dependencies between modules.

11. Component Diagram

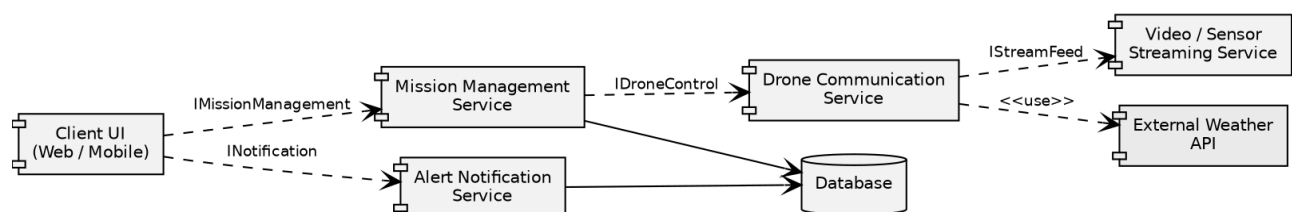


Fig. 7 — Component Diagram showing the system's services and their interfaces.

12. Deployment Diagram

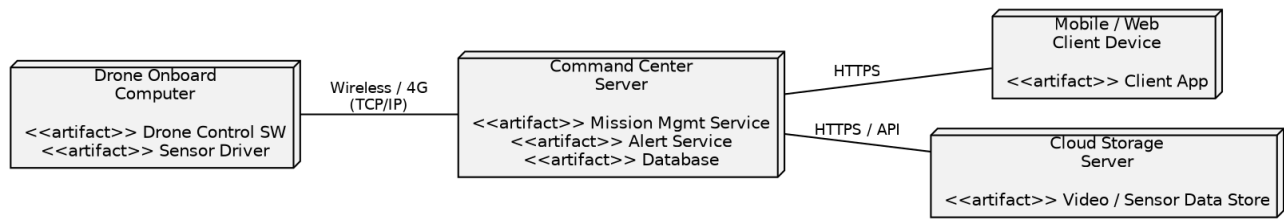


Fig. 8 — Deployment Diagram showing hardware nodes and how they communicate.

13. Object Responsibility Analysis

Class	Responsibility
EmergencyCoordinator	Owens the decisions behind incident registration and mission assignment.
Drone	Tracks its own status and carries out flight commands.
RescueMission	Brings together the drone, route and team for a single mission.
Alert	Holds alert data and notifies the right rescue team.
Zone	Keeps track of its own risk level and the victims located within it.

14. Design Axioms, Principles and Design Pattern Justification

Design Axioms

- Independence Axiom: each class is responsible for one concern only (Alert, for instance, handles notification and nothing about mission logic), which keeps coupling low.
- Information Axiom: the design favours simplicity, with each class given a small, clearly defined set of responsibilities.

Design Principles Applied

- High cohesion: attributes and methods within a class are closely related to one another.
- Low coupling: classes communicate through method calls rather than sharing internal data directly.
- Encapsulation: attributes are kept private and exposed only through public accessor methods.
- Abstraction: the User superclass hides login logic that its subclasses share.

Design Patterns Used

- Observer — Alert notifies the Rescue Team objects subscribed to it whenever it is raised.
- State — a Drone's behaviour depends on which lifecycle state it is currently in.
- Strategy — route planning can switch between shortest-path and safest-path algorithms.
- Singleton — one MissionControlManager instance coordinates every active mission.

15. Assumptions and Constraints

Assumptions

- Drones carry onboard GPS and maintain stable telemetry links under normal operating conditions.
- Every operator and coordinator using the system is trained and authorised.

- Sensor and video data follow a standard format across all drones.

Constraints

- The system needs to deliver real-time updates with minimal delay.
- Every UML diagram must stay consistent with the use cases it is based on.
- Access control and security must be applied to all user actions.

16. Requirement-to-Use Case / Class Traceability

Requirement	Use Case	Class(es)
Register incidents	Register Disaster Incident	DisasterIncident, EmergencyCoordinator
Assign drones to missions	Assign Drone to Mission	RescueMission, Drone
Monitor drones in real time	Monitor Drone Status	Drone, DroneOperator
Detect victims	Detect Victim Location	SensorData, CameraFeed, Victim
Notify rescue teams	Generate Emergency Alert	Alert, RescueTeam

17. Implementation / Prototype and Results

This submission is a design-and-analysis deliverable rather than a working build. The UML diagrams shown above were produced with standard diagramming tools (comparable to StarUML or Draw.io) and checked for consistency against the use cases and class model.

A working implementation would build out the User hierarchy, the Drone state machine and the Alert/Observer notification mechanism in Python or Java, with a relational database backing incidents, missions and drones.

18. Reflection on Design Decisions and Learning Outcomes

Working through the ADDRRS design made clear how much OOAD decisions shape real-world outcomes: the Drone state machine needed to match genuine flight constraints, and using the Observer pattern kept alert handling separate from mission logic. Weighing drone autonomy against the need for human oversight was one of the harder trade-offs in the design.

The project ties into SDG 3 (rapid medical response), SDG 9 (resilient infrastructure), SDG 11 (safer communities) and SDG 13 (climate-related disaster resilience).

19. GitHub Upload

<https://github.com/ShaikIrfan19/CSA1109-OOAD>